

Contents

1	DD08: トークン発行・検証	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 用途別 TTL 一覧	2
1.4.2	3.2 HMAC-SHA256 保存方式	2
1.4.3	3.3 一回限りトークン	2
1.4.4	3.3a メンバー招待トークン(activation)+ SCR-031 有効化	3
1.4.5	3.3b プロジェクト連絡先メール確認トークン(contact_verify)+ SCR-032 着地	3
1.4.6	3.4 再入室トークン	4
1.4.7	3.5 期限切れトークン削除(cron)	4
1.5	4. 関連設計	4
1.6	5. テスト観点	4
1.7	6. 未確定事項・確認事項	4

1 DD08: トークン発行・検証

1.1 0. 文書情報

項目	内容
文書名	DD08: トークン発行・検証
詳細設計 ID	DD08
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-003 (メール確認) / FR-004 / FR-006 (パスワード再設定) / FR-083 / FR-084 (再入室) / FR-018 (管理者ユーザー有効化)
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-003	メール確認トークン
機能	FR-004 / FR-006	パスワードリセットトークン
機能	FR-018	管理者ユーザー有効化(activation) トークン
機能	FR-083 / FR-084	再入室トークン (30 日、SCR-027)
API	/me/email-verification/* /auth/password-reset/admin-users/{id}/activation /widget/v1/reentry/*	トークン発行・検証 API
テーブル	access_tokens	トークン基盤

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 10.5.1	用途別 TTL 一覧	4 用途 × TTL / 一回限り / 内包データ
§ 10.5.2	HMAC-SHA256 保存方式	rawToken 返却 / hash 列保存 / verify / consume
§ 10.5.3	一回限りトークン	consumeToken() 強制
§ 10.5.4	再入室トークン	30 日 / 部屋 reopen 時の新トークン発行

1.4 3. 詳細設計本文

1.4.1 3.1 用途別 TTL 一覧

用途	TTL	一回限り	内包データ
email_verify	24h	◎	userId
password_reset	24h	◎	userId
activation	7d	◎	userId, contractOwnerUserId + meta.invitedProjectId / meta.invitedRole(JSON)
reentry	30d	×	inquiryId
contact_verify	24h	◎	meta.projectId(JSON)(SCR-032 着地時に projects.contact_email_verified_at を更新する対象プロジェクトを特定。user_id は持たない —— 連絡先メール所有者がアカウントを持つとは限らない)

1.4.2 3.2 HMAC-SHA256 保存方式

```
// app/shared/src/lib/token.ts
export async function generateToken(
  purpose: TokenPurpose, payload: Record<string, string>, env: Env, ttlSec: number,
): Promise<string> {
  const tokenId = generateUlid();
  const random = crypto.getRandomValues(new Uint8Array(32));
  const rawToken = `${tokenId}.${base64urlEncode(random)}`;
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);

  await env.DB.prepare(`
    INSERT INTO access_tokens (id, token_hash, purpose, meta, created_at, expires_at, user_id)
    VALUES (?, ?, ?, ?, ?, ?, ?)
  `).bind(tokenId, tokenHash, purpose, JSON.stringify(payload),
    new Date().toISOString(),
    new Date(Date.now() + ttlSec * 1000).toISOString(),
    payload.userId ?? null).run();

  return rawToken; // ユーザーに返す生トークン (DB には hash のみ)
}

export async function verifyToken(
  rawToken: string, purpose: TokenPurpose, env: Env,
): Promise<{ payload: Record<string, string> }> {
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);
  const record = await env.DB.prepare(
    `SELECT id, purpose, meta, expires_at, used_at FROM access_tokens WHERE token_hash = ?1`
  ).bind(tokenHash).first<TokenRow>();
  if (!record) throw new HTTPException(400, { message: 'TOKEN_INVALID' });
  if (record.purpose !== purpose) throw new HTTPException(400, { message: 'TOKEN_INVALID' });
  if (record.used_at) throw new HTTPException(400, { message: 'TOKEN_REUSED' });
  if (new Date(record.expires_at).getTime() < Date.now()) {
    throw new HTTPException(400, { message: 'TOKEN_EXPIRED' });
  }
  return { payload: JSON.parse(record.meta) };
}

export async function consumeToken(rawToken: string, purpose: TokenPurpose, env: Env) {
  const tokenHash = await hmacSha256(env.MASTER_KEY, rawToken);
  await env.DB.prepare(
    `UPDATE access_tokens SET used_at = ?1 WHERE token_hash = ?2`
  ).bind(new Date().toISOString(), tokenHash).run();
}
```

1.4.3 3.3 一回限りトークン

consumeToken() を検証直後に必ず呼び、used_at を設定。再使用時は TOKEN_REUSED。

1.4.4 3.3a メンバー招待トークン (activation)+ SCR-031 有効化

招待時 (POST /projects/{id}/members) に purpose='activation' で発行し、meta に invitedProjectId / invitedRole を JSON 保持。SCR-031 「登録を完了する」 押下時に verifyToken → 同一トランザクションでユーザー有効化 + consumeToken を実行する。

```
// 招待時 (POST /projects/{id}/members)
const rawToken = await generateToken('activation', {
  userId,
  contractOwnerUserId,
  invitedProjectId: projectId,
  invitedRole,           // 'admin' | 'member'
}, env, /* 7d */ 7 * 86400);
const activationUrl = `${env.WEB_BASE}/auth/activate?token=${encodeURIComponent(rawToken)}`;
await enqueueEmail({ template: 'TPL-ADMIN_USER_REGISTER', activationUrl, ... });

// SCR-031 で「登録を完了する」 押下時 (POST /auth/invitations/{token}/activate)
const { payload } = await verifyToken(rawToken, 'activation', env);
const { userId, invitedProjectId, invitedRole } = payload;
await env.DB.batch([
  env.DB.prepare(
    `UPDATE users SET name = ?1, password_hash = ?2, status = 'active', updated_at = ?3 WHERE id = ?4`
  ).bind(displayName, passwordHash, now, userId),
  env.DB.prepare(
    `INSERT INTO terms_agreements (user_id, terms_version, privacy_version, agreed_at) VALUES (?, ?, ?, ?)`
  ).bind(userId, termsVer, privacyVer, now),
  env.DB.prepare(
    `UPDATE project_users SET valid = 1, updated_at = ?1 WHERE user_id = ?2 AND project_id = ?3`
  ).bind(now, userId, invitedProjectId),
]);
await consumeToken(rawToken, 'activation', env);
await writeAudit({ action: 'user.activation_completed', actor_user_id: userId, target_user_id: }
```

再送時 (POST /members/{id}/resend-invitation) は旧トークンを consumeToken() で失効させ、同一 invitedProjectId / invitedRole を meta に含めて新規トークンを発行する。

1.4.5 3.3b プロジェクト連絡先メール確認トークン (contact_verify)+ SCR-032 着地

SCR-010-M1 プロジェクト設定モジュールで連絡先メールが新規入力 / 変更された時に purpose='contact_verify' で発行し、meta に projectId を JSON 保持。SCR-032 着地ページから POST /auth/contact-verifications/{token} 経由で同一トランザクションで projects.contact_email_verified_at セット + consumeToken を実行する。

```
// 連絡先メール変更時 (POST/PATCH /projects/{id} で contact_email が変更された場合)
const rawToken = await generateToken('contact_verify', { projectId }, env, /* 24h */ 24 * 3600);
const verifyUrl = `${env.WEB_BASE}/auth/contact-verify?token=${encodeURIComponent(rawToken)}`;
await enqueueEmail({ template: 'TPL-PROJECT_CONTACT_VERIFY', verifyUrl, ... });

// SCR-032 着地時 (POST /auth/contact-verifications/{token})
const { payload } = await verifyToken(rawToken, 'contact_verify', env);
const { projectId } = payload;
await env.DB.prepare(
  `UPDATE projects SET contact_email_verified_at = ?1, updated_at = ?1 WHERE id = ?2`
).bind(now, projectId).run();
await consumeToken(rawToken, 'contact_verify', env);
await writeAudit({ action: 'project.contact_email_verified', target_project_id: projectId, meta: }
```

Turnstile 不要 (状態変更は単純な verified_at セットのみで副作用が軽微、トークン HMAC-SHA256 検証 + IP/トークン単位のレート制限で十分)。再送は連絡先メール再設定 / 再保存時の自動発行のみで、明示的な「再送」操作 API は提供しない (MVP)。

1.4.6 3.4 再入室トークン

```
// 再入室トークンは長寿命のため、access_tokens テーブルに `meta.inquiryId` を内包
// 失効・ローテーション運用:
// - チャット部屋作成時に発行
// - 30 日経過で expires_at に達し失効
// - 部屋を closed → reopen した時に新トークン発行 (旧トークンは expires_at まで有効)
```

1.4.7 3.5 期限切れトークン削除 (cron)

`access_tokens` の `expires_at < NOW() - 30` 日を物理削除。詳細は [DD14_ バッチ・非同期処理.md](#) §14.1.9 を参照。

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
認証・認可設計	../02_ 基本設計/08_ 認証・認可設計.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD01_ アカウント・ユーザー管理.md / DD06_ 個別チャット.md / DD11_ 暗号化・管理.md

1.6 5. テスト観点

観点	内容
単体	generateToken / verifyToken / consumeToken の正常系 + 異常系全網羅
結合	メール確認 / パスワード再設定 / 管理者ユーザー activation の全 4 用途で発行 → 検証 → consume
異常系	TOKEN_INVALID / TOKEN_REUSED / TOKEN_EXPIRED / TOKEN_PURPOSE_MISMATCH
境界値	TTL ぴったりの境界 (24h / 1h / 7d / 30d) / rawToken の base64url 形式
セキュリティ	DB には hash のみ保存 (生トークン非保存) / HMAC-SHA256 + MASTER_KEY 派生
性能	verifyToken は単一 SELECT、p95 < 50ms

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済